DOSSIER DE GOUVERNANCE

Continuité, risques et conformité

BIA • PCA/PRA • EBIOS RM ateliers 2 à 5 • NIS2/DORA • Recommandations

0 L'essentiel en une page

Ce dossier couvre la partie **gouvernance** exigée par le sujet (section 3.5) : ce que coûte un arrêt (BIA), comment on reprend (PCA/PRA), l'analyse de risques EBIOS RM (ateliers 2 à 5 et mise à jour de l'atelier 1), la conformité NIS2/DORA et nos recommandations à la direction.

Tous les chiffres de reprise ont été mesurés le 20/10/2026 sur la stack réelle (17 services Docker), en provoquant de vraies pannes. Ils sont à refaire sur le PC Hôte avant la soutenance.

Ce qu'on a mesuré	Résultat	Ce que ça veut dire
Crash de l'API de paiement	**reprise en 1,8 s**	Docker relance le conteneur tout seul
Crash de la base PostgreSQL	**reprise en 0,4 s, 0 transaction perdue**	PostgreSQL rejoue son journal (WAL) au redémarrage
Procédure Ansible restore_database	**9,8 s**, contrôles OK	Contrôle d'intégrité, relance de l'API, paiement de preuve
Flood de Clara (400 requêtes, 730/s)	**90,5 % rejetées**, service maintenu	La limitation de débit absorbe l'attaque…
Client légitime extérieur pendant le flood	**3 paiements sur 25 refusés**	… mais touche aussi les vrais clients (même IP vue par le WAF)

À retenir : 1 minute d'arrêt coûte environ **346 € de commissions** et bloque **28 800 € de paiements** pour nos marchands. Nos reprises mesurées se comptent en secondes. Notre point faible n'est pas la vitesse de reprise mais ce qui n'existe pas encore : la **réparation pilotée par la détection**, les **sauvegardes** et l'**authentification**.

1 Analyse d'impact (BIA) : combien coûte un arrêt des paiements

Les chiffres de l'entreprise sont **fictifs** (le sujet l'impose) mais réalistes pour un prestataire de paiement de taille moyenne. Ils servent à fixer nos objectifs de reprise et à relier la gravité EBIOS à un coût.

1.1 Hypothèses

Hypothèse	Valeur	Commentaire
Volume moyen en journée	600 paiements / min	Pic : 1500 / min (soldes, fin de mois)
Montant moyen d'un paiement	48 €	Panier e-commerce moyen
Commission perçue	1.2 %	Notre chiffre d'affaires sur chaque paiement
Pénalité partenaires (SLA 99,9 %)	5 000 € / heure entamée	Au-delà de 15 min d'arrêt
Cellule de crise	4 personnes x 90 €/h	Mobilisée au-delà de 15 min

1.2 Coût selon la durée de l'arrêt

Durée	Paiements bloqués	Flux bloqué (marchands)	Commissions perdues	Pénalités	Crise	Coût pour nous	Gravité EBIOS
2 s (crash API mesuré)	20	960 €	12 €	—	—	**12 €**	G1
2 min (RTO cible)	1 200	57 600 €	691 €	—	—	**691 €**	G2
15 min	9 000	432 000 €	5 184 €	—	90 €	**5 274 €**	G3
1 h	36 000	1 728 000 €	20 736 €	5 000 €	360 €	**26 096 €**	G3
4 h	144 000	6 912 000 €	82 944 €	20 000 €	1 440 €	**104 384 €**	G4

- **1 minute d'arrêt** = 346 € de commissions perdues et 28 800 € de paiements bloqués (au pic : 864 € et 72 000 €).
- **Au-delà de 15 min**, les pénalités partenaires et la cellule de crise s'ajoutent : le coût change d'échelle.
- **Le vrai risque n'est pas que financier** : un arrêt long est un incident majeur au sens de DORA (notification à l'autorité), et les marchands peuvent partir chez un concurrent. Ces impacts ne sont pas chiffrés ici.

1.3 Objectifs de reprise retenus

Objectif	Cible	Justification	Mesuré aujourd'hui
RTO : durée maximale d'arrêt	**2 min**	Coût encore limité (~700 €), avant les pénalités	1,8 s (crash API), 0,4 s (crash base)
RPO : perte de données maximale	**0 paiement validé**	Un paiement confirmé perdu = litige client	0 perte sur crash de la base
Temps de détection	< 30 s	Laisser le temps de réparer et vérifier dans le RTO	Non mesurable : détection pas encore branchée

2 Plan de continuité et de reprise (PCA/PRA)

Le **PCA** décrit comment le service **continue** pendant l'incident, le **PRA** comment on le **rétablit** ensuite. Chaque procédure s'appuie sur un composant qui existe dans le dépôt.

2.1 Scénarios couverts

Scénario	Ce qui se passe	Réponse (automatique ou procédure)	Résultat mesuré	Couvert ?
S1 - Flood sur /api/pay (ER1)	Attaque depuis le PC Attaquant (red_attack.py --mode flood)	Limitation de débit dans l'API (40 requêtes / 2 s par client)	90,5 % de l'attaque rejetée, service maintenu. 3/25 clients légitimes refusés	**Partiel**
S2 - Crash de l'API	Le processus de payment-api s'arrête	Docker relance le conteneur (restart: unless-stopped)	Reprise en 1,8 s	**Fait**
S3 - Crash de la base	Arrêt brutal de PostgreSQL	Docker relance PostgreSQL, rejeu du journal (WAL)	Reprise en 0,4 s, 0 transaction perdue	**Fait**
S4 - Doute sur l'intégrité des données	Après un incident, il faut prouver que les comptes sont justes	Playbook *restore_database*	9,8 s, contrôles OK, paiement de preuve accepté	**Fait**
S5 - Source malveillante identifiée	Un outil d'attaque reconnaissable (User-Agent)	Playbook *block_source* (règle dans le WAF)	Règle appliquée, mais le WAF redémarre (coupure de quelques secondes)	**Partiel**
S6 - Service dégradé sans crash	L'API répond mal mais ne s'arrête pas	Playbook *restart_service*, à déclencher par la détection	Le playbook fonctionne ; le déclenchement automatique n'existe pas	**Partiel**
S7 - Perte du PC Hôte ou du disque	Matériel en panne, volume de données perdu	Aucune : pas de sauvegarde, pas de second PC	Non couvert	**Absent**

2.2 Procédures de reprise (playbooks Ansible)

Les playbooks tournent dans le conteneur *remediation*. Ils ne peuvent agir sur Docker qu'à travers un proxy qui n'autorise **que la lecture et le redémarrage**, et seulement sur une liste blanche de services (jamais la base, la supervision ou les proxys).

Playbook	Déclencheur	Étapes	Preuve de reprise	Durée
restart_service	Service dégradé (S2, S6)	1. Vérifie que le service est autorisé 2. Le redémarre 3. Attend qu'il soit sain (60 s max)	Paiement de test via le WAF	quelques s
block_source	Source malveillante (S5)	1. Contrôle la valeur (refuse les IP Docker) 2. Écrit la règle (IP, User-Agent ou limitation) 3. Recharge le WAF	Requête de contrôle bloquée (403)	quelques s
restore_database	Doute sur les données (S4)	1. Contrôles d'intégrité 2. Redémarre l'API 3. Paiement de test 4. Nouveaux contrôles	Contrôles OK avant et après + paiement accepté	9,8 s

Attention : Aujourd'hui ces playbooks se lancent à la main (make restart, make block, make restore). Pour le jury, ils doivent être lancés **par le moteur de remédiation** à partir de la détection, sinon c'est une intervention humaine (0/5 sur l'axe 1).

2.3 Qui fait quoi (RACI)

R = réalise, **A** = rend des comptes (un seul par ligne), **C** = consulté, **I** = informé.

Activité	Moteur auto	Antoine (Dev)	Mélanie (Infra)	Clara (Cyber)	Agathe (IA)	Timéo (Gouv.)
Détecter l'anomalie	R	I	C	C	A	I
Décider de la réparation	R	A	C	C	C	I
Redémarrer / restaurer (playbooks)	R	C	A	I	—	I
Bloquer une source dans le WAF	R	I	C	A	—	I
Vérifier par une transaction test	R	A	C	—	—	I
Rotation des secrets (fuite)	—	C	C	A/R	—	I
Communication, notification DORA	—	C	C	C	C	A/R
Retour d'expérience (post-mortem)	—	R	R	R	R	A

2.4 Risques résiduels après reprise

Risque résiduel	Pourquoi	Décision
Un seul PC Hôte	Tout tourne sur une machine : panne matérielle = arrêt total (S7)	Accepté pour le prototype, à traiter (R2)
Pas de sauvegarde de la base	restore_database contrôle et relance, mais ne restaure rien. Volume perdu = données perdues	Accepté pour le prototype, à traiter (R2)
Blocage par IP inutilisable	Sous Docker Desktop, le WAF voit tous les clients extérieurs avec la même IP (10.77.10.1). Le playbook refuse donc de bloquer ces adresses	Bloquer par User-Agent ou par limitation de débit (R3)
Effet collatéral de la limitation de débit	Pendant un flood, des clients légitimes extérieurs sont refusés (3/25 mesurés)	Accepté temporairement, à traiter (R3)
Limitation contournable	Elle se fie à l'en-tête X-Forwarded-For, que l'attaquant peut falsifier (0 requête bloquée sur 60 en changeant d'IP)	À corriger avant jeudi (Clara)
Boucle de réparation non finie	Les playbooks existent mais rien ne les déclenche à partir de la détection ML	Priorité n°1 de l'équipe (R1)
Blocage = redémarrage du WAF	Appliquer une règle coupe le service quelques secondes pour tout le monde	À corriger (rechargement sans redémarrage)

3 EBIOS RM : ateliers 2 à 5

Version courte, reliée au démonstrateur. Elle reprend les sources de risque du rapport de Clara pour éviter les doublons.

3.1 Atelier 2 : qui pourrait attaquer, et pourquoi

Source de risque	Ce qu'elle veut (objectif visé)	Motivation	Ressources	Retenue ?
SR1 - Cybercriminel	Rendre les paiements indisponibles pour exiger une rançon	Forte (argent)	Moyennes (outils de flood)	Oui : scénario de la démo
SR2 - Fraudeur	Passer des paiements frauduleux, rejouer des transactions	Forte (argent)	Faibles à moyennes	Oui
SR3 - Initié ou dépendance piégée	Voler les secrets ou les données clients	Moyenne	Élevées (accès interne, chaîne logicielle)	Oui
Hacktiviste	Nuire à l'image de l'entreprise	Faible pour un acteur du paiement	Variables	Non (pertinence faible)

3.2 Atelier 3 : par où passerait l'attaquant (scénarios stratégiques)

Partie prenante	Pourquoi elle compte	Niveau de confiance
Clients (navigateurs)	Seul accès autorisé à la plateforme, via le WAF	Faible : n'importe qui
Marchands partenaires	Reçoivent les paiements, subissent l'arrêt	Moyen
Équipe d'exploitation	Détient la clé SOPS et l'accès au PC Hôte	Élevé mais risque d'erreur ou de malveillance
Fournisseurs logiciels	Images Docker, paquets Python et npm, modèle d'IA téléchargé au démarrage	Moyen : versions figées pour les images, pas pour tout

- **Scénario stratégique 1** (SR1 → ER1) : le cybercriminel sature l'API de paiement depuis Internet ; les marchands ne sont plus payés.
- **Scénario stratégique 2** (SR2 → ER2, ER3) : le fraudeur passe des paiements sans être authentifié ou rejoue une transaction.
- **Scénario stratégique 3** (SR3 → ER4, ER6) : une dépendance piégée ou un initié vole les secrets, puis détourne le moteur de réparation.

3.3 Atelier 4 : scénario opérationnel n°1, le flood (attaque de Clara)

Étape	Ce que fait l'attaquant	Ce que fait la plateforme (constaté)
1. Repérage	Trouve la page publique et la route /api/pay	Le WAF ne laisse passer que 3 routes de l'API (le reste répond 403)
2. Attaque	Lance red_attack.py --mode flood : 400 requêtes, 25 en parallèle, 730 requêtes/s	La limitation de débit répond 429 à 90,5 % des requêtes
3. Effet	Cherche à saturer l'API et la base	Le service reste disponible ; 3 clients légitimes sur 25 refusés au passage
4. Contournement possible	Change de fausse IP à chaque requête (en-tête X-Forwarded-For)	Plus aucune requête bloquée : la protection est contournée
5. Réparation attendue	—	Détection ML puis playbook : **pas encore branché**

Vraisemblance : V3 (très vraisemblable). L'outil est simple et le contournement aussi. **Gravité : G3** (paiements bloqués au-delà du RTO si la protection est contournée). Le scénario n°2 (fuzzing, SC-02 du rapport de Clara) est traité par la validation des données de l'API.

3.4 Atelier 5 : plan de traitement

Risque	Mesure	Responsable	Échéance	Risque résiduel
ER1 Flood	Ne plus faire confiance à X-Forwarded-For dans le WAF ; brancher la détection ML sur les playbooks	Clara, Antoine, Agathe	Jeudi 17 h	Effet collatéral sur les clients extérieurs (Docker Desktop)
ER2 Paiement faux	Débiter le compte dans la même requête que l'enregistrement ; rendre la clé d'idempotence obligatoire	Antoine	Après le workshop	Accepté pour le prototype
ER3 Compte client	Mettre en service le conteneur auth (aujourd'hui provisoire)	Antoine	Après le workshop	Paiements sans authentification : à dire au jury
ER4 Secrets	Passer des variables d'environnement aux fichiers /run/secrets	Clara	Si le temps le permet	Secrets visibles avec docker inspect
ER6 Moteur	Protéger les routes /waf/block et /waf/unblock ; garder la liste blanche Ansible	Clara, Mélanie	Jeudi 17 h	Faible
ER7 PC Hôte	Sauvegarde quotidienne de la base hors du PC + test de restauration	Mélanie	Après le workshop	Accepté pour le prototype

4 Atelier 1 mis à jour : ce qui nous protège vraiment

La version du 19/10 annonçait des protections prévues dans l'architecture. Voici ce qui existe **réellement** dans le code au 20/10, vérifié sur la stack. Cette version remplace la colonne « Ce qui nous protège déjà ».

Risque	Gravité	Protection annoncée le 19/10	État réel au 20/10	Statut
ER1 Les paiements ne passent plus	G3	WAF qui limite le débit, détection ML, IP bannie	Limitation de débit active (90,5 % du flood rejeté) mais contournable ; ML pas branché ; bannir une IP impossible sous Docker Desktop	**Partiel**
ER2 Un paiement est faux	G4	Paiement en une requête avec contrôle du solde	Contrainte « solde positif » et unicité de la clé d'idempotence en base (vérifiées). Mais le paiement ne débite aucun compte et la clé est facultative	**Partiel**
ER3 Un pirate entre dans un compte	G3	Essais limités sur /login, détection ML	Le service auth est un conteneur provisoire : il n'y a ni connexion ni compte client	**Absent**
ER4 Vol de nos secrets	G4	Secrets chiffrés SOPS, clé JWT seulement dans auth	SOPS + age en place, .env hors de Git. Secrets en variables d'environnement ; pas de JWT (auth absent)	**Partiel**
ER5 Fuite des données clients	G3	Règles anti-injection du WAF, base inaccessible	WAF : 3 routes autorisées ; base non publiée (après correction du compose, qui la publie encore sur main) ; erreurs masquées (CWE-209)	**Fait**
ER6 Moteur de réparation détourné	G4	Accès Docker limité, aucune commande de l'extérieur	Proxy Docker limité au redémarrage + liste blanche Ansible. Mais routes /waf/block sans authentification sur le réseau interne ; moteur pas construit	**Partiel**
ER7 Le PC Hôte tombe	G3	Playbook pour tout réinstaller	Relance automatique des conteneurs (mesurée). Pas de sauvegarde, pas de second PC	**Partiel**

5 Conformité NIS2 / DORA

Pour une entité financière, **DORA** (règlement (UE) 2022/2554) s'applique en priorité : c'est la règle spécifique au secteur, NIS2 (directive (UE) 2022/2555) reste la référence générale. Ce tableau fusionne la section DORA du rapport de Clara avec l'analyse de gouvernance.

Exigence DORA	Ce que le démonstrateur prouve	Ce qu'il ne peut pas prouver	Statut
Cadre de gestion du risque TIC (art. 5 à 9)	Inventaire des actifs (architecture, plan d'adressage), analyse EBIOS RM, durcissement des conteneurs, réseaux isolés	Implication de l'organe de direction, politique formelle validée	**Partiel**
Détection (art. 10)	Modèle Isolation Forest entraîné sur du trafic réel ; supervision Prometheus, Loki, Grafana	Détection branchée en continu sur la supervision	**Partiel**
Réponse et rétablissement (art. 11)	Reprises mesurées (1,8 s et 0,4 s), playbooks avec preuve par transaction	Déclenchement automatique de bout en bout	**Partiel**
Sauvegarde et restauration (art. 12)	Aucune perte sur crash de la base (journal WAL)	Sauvegardes, restauration depuis une copie, site de secours	**Absent**
Gestion et classification des incidents (art. 17 et 18)	Journal d'audit en base (service_audit_logs), journaux centralisés dans Loki	Processus de classification, critères d'incident majeur	**Partiel**
Notification des incidents majeurs (art. 19)	Les données nécessaires existent (heure, durée, impact mesuré)	Notification réelle à l'autorité (en France : l'ACPR)	**Absent**
Tests de résilience (art. 24 à 27)	Tests de panne réels (crash API et base), attaques simulées, make chaos	Tests de pénétration avancés (TLPT) par un tiers	**Partiel**
Risque lié aux prestataires TIC (art. 28 et suivants)	Versions d'images Docker figées	Registre des prestataires, clauses contractuelles, stratégie de sortie	**Absent**
Partage d'informations (art. 45)	—	Participation à des échanges sur les menaces	**Absent**

Côté NIS2, les mesures de l'article 21 (analyse de risques, gestion des incidents, continuité, sécurité de la chaîne d'approvisionnement, contrôle d'accès) recoupent le tableau ci-dessus. L'obligation de notification (article 23 : alerte précoce sous 24 h, notification sous 72 h, rapport final sous un mois) n'est pas démontrable avec un prototype.

Attention : Corrections à reporter dans le rapport de Clara : le blocage **redémarre** le WAF ; la clé d'idempotence est **facultative** ; la limitation « par IP » est **contournable** ; le RTO de 1,8 s vient du **redémarrage automatique de Docker**, pas de la réparation pilotée par le ML.

6 Recommandations à la direction

Ce que nous ferions dans un contexte industriel, par ordre de priorité. Chaque recommandation répond à un risque mesuré.

#	Recommandation	Pourquoi (preuve)	Effort	Indicateur de succès
R1	**Industrialiser la réparation pilotée par la détection** : la supervision alimente le ML, qui déclenche les playbooks, qui prouvent la reprise	Les playbooks marchent (9,8 s) mais rien ne les déclenche : aujourd'hui la reprise d'une attaque dépend d'un humain	Moyen (déjà 80 % des briques)	Incident détecté et réparé sans humain en moins de 2 min, mesuré en test mensuel
R2	**Sauvegarder la base hors du serveur et ajouter un second site**	Une panne du PC ou du disque = perte totale (S7). DORA art. 12 non couvert	Moyen à élevé (budget infra)	RPO de 15 min garanti, restauration testée chaque trimestre
R3	**Placer le pare-feu en frontal et vérifier l'identité des clients** (service d'authentification, MFA)	Le WAF ne voit pas les vraies IP, la limitation se contourne et touche les vrais clients (3/25)	Moyen	0 client légitime refusé pendant un flood ; contournement impossible
R4	**Formaliser la gestion des incidents DORA** : classification, notification, exercices de crise	Les données existent mais aucun processus n'existe pour notifier dans les délais	Faible	Exercice de crise semestriel, notification simulée en moins de 4 h
R5	**Maîtriser la chaîne logicielle** : inventaire des composants (SBOM), analyse des images, registre des prestataires	Le service IA télécharge un modèle sur Internet à chaque démarrage ; pas de registre des fournisseurs	Faible à moyen	100 % des images analysées avant déploiement

Arbitrage proposé : R1 et R4 d'abord (peu coûteux, fort impact sur la conformité DORA), puis R3, puis R2 qui demande un budget d'infrastructure.